**Evaluación de Arquitectura de software Nº2**

Integrantes: Jhonatan De La Cruz Tigre, Lorenzo Cotta

Profesor: Pedro Pablo Concepcion

**Proyecto:** Modernización Plataforma Core *NexusEngine v2.0* — NexusCorp Omni-Logistics

# SECCIÓN 1: Visión del Sistema y Contexto de Negocio

## 1.1. Propósito y Objetivos Estratégicos de Negocio (Business Drivers)

Actualmente, NexusCorp procesa más de 120.000 despachos diarios en 5 países sobre *NexusEngine*, un sistema monolítico en Ruby y PostgreSQL que presenta serios problemas de acoplamiento y deuda técnica. Con el rediseño hacia la nueva arquitectura (*NexusEngine v2.0*), buscamos eliminar las caídas generales del sistema y sostener el crecimiento del negocio a través de cuatro objetivos concretos:

1. **Continuidad Operativa y Alta Disponibilidad:** Evitar las caídas totales del sistema en eventos de alta demanda como *CyberMonday* (donde 4 horas de inactividad generaron pérdidas por USD 350.000), asegurando una disponibilidad mínima (SLA) >= 99.95% anual.

2. **Escalabilidad y Bajo Tiempo de Respuesta:** Soportar picos de concurrencia de 15.000 solicitudes por segundo (RPS) manteniendo tiempos de respuesta en el percentil 95 (p95) menores a 250 ms tanto al consultar el seguimiento de envíos (*tracking*) como al registrar nuevos pedidos.

3. **Facilidad de Integración y Mantenibilidad:** Reducir el tiempo necesario para conectar nuevas empresas transportistas externas o pasarelas de pago de varias semanas a menos de 5 días hábiles, mediante interfaces desacopladas sin tener que modificar ni reiniciar el núcleo del sistema.

4. **Seguridad e Integridad en Pagos:** Cumplir con la norma PCI-DSS para el manejo de billeteras digitales y pagos contra entrega, protegiendo las comunicaciones con cifrado y manteniendo un registro de auditoría inalterable para las conciliaciones bancarias.

## 1.2. Identificación de Stakeholders y Preocupaciones (Concerns)

| Stakeholder | Rol en la Organización / Ecosistema | Preocupaciones Arquitectónicas | ASRs Relacionados |
|---|---|---|---|
| Gerencia y Directorio de Negocio | Patrocinadores del proyecto y encargados de las decisiones estratégicas. | Costos, retorno de inversión, evitar pérdidas millonarias por caídas en CyberMonday (USD 350.000) y permitir el crecimiento en los 5 países. | ASR-01, ASR-02, ASR-03 |
| Clientes B2B (Comercios) y B2C (Usuarios Finales) | Usuarios que registran órdenes de despacho y consultan el estado de sus envíos. | Disponibilidad del servicio y tiempos de respuesta rápidos al crear pedidos y revisar el seguimiento en línea sin errores del servidor (HTTP 5xx). | ASR-01, ASR-02 |
| Transportistas (Conductores en Ruta) | Conductores que reciben rutas, reportan ubicación GPS y registran entregas y cobros. | Operación continua desde la aplicación móvil; que un fallo en facturación o en un banco nunca bloquee sus rutas ni el registro de entregas. | ASR-01, ASR-02 |
| Equipo de Finanzas, Contabilidad y Auditoría | Responsables de conciliación bancaria, cobros, billeteras digitales y facturación tributaria. | Integridad de los cobros (evitar cobros duplicados o pérdidas), registros contables que no se puedan alterar y cumplimiento PCI-DSS. | ASR-01, ASR-04 |
| Equipo de Desarrollo | Ingenieros encargados de construir, mantener y evolucionar la plataforma | Mantenibilidad, claridad de la arquitectura, eliminar el módulo gigante `OrderManager` (14.000 líneas de código), lograr bajo acoplamiento y facilitar las pruebas | ASR-03 |
| Equipo de Operaciones e Infraestructura TI | Administradores de los servidores en la nube, despliegues y monitoreo | Disponibilidad, monitoreo, escalamiento horizontal ante picos de tráfico y capacidad de desplegar cambios en un servicio sin bajar toda la plataforma | ASR-01, ASR-02, ASR-03 |
| Proveedores Externos (Transportistas y Pasarelas de Pago) | Sistemas externos de transporte internacional y procesadores bancarios | Interoperabilidad mediante interfaces y APIs estándar, transmisión segura de datos (TLS 1.3) y manejo confiable de solicitudes | ASR-03, ASR-04 |

# SECCIÓN 2: Escenarios de Atributos de Calidad Formales (Plantilla SEI)

A continuación, se modelan formalmente los cuatro escenarios de atributos de calidad vinculados directamente a los ASRs de NexusCorp:

**Escenario 1: ASR-01 — Alta Disponibilidad y Resiliencia Operativa (Tolerancia a Fallos y Degradación Controlada)**

- **Atributo de Calidad:** Disponibilidad / Tolerancia a Fallos.
- **Fuente del Estímulo:** Sistema externo de Pasarela Bancaria o Servicio Interno de Facturación.
- **Estímulo:** Caída total (error HTTP 503 o tiempo de espera agotado) del servidor de facturación o del banco externo por saturación.

- **Artefacto Afectado:** Servicio de Órdenes (*Order Service*), Servicio de Despacho y Broker de Mensajes (Kafka).

- **Entorno Operacional:** Operación normal y alta demanda (120.000 despachos diarios en 5 países con transportistas en ruta).

- **Respuesta Esperada del Sistema:** El servicio de órdenes aplica tolerancia a fallos aislando la caída de facturación. Las órdenes nuevas y las entregas en ruta se registran normalmente en la base de datos con estado PENDIENTE_FACTURACION y se publica un evento en el broker de mensajes (Kafka) para procesar la factura de forma asíncrona mediante una cola de reintentos cuando el servicio se recupere, sin detener la operación logística.

- **Medida Cuantitativa de la Respuesta:**
  - Disponibilidad mayor o igual al 99.95% anual en registro de pedidos y rutas.
  - 0% de pedidos rechazados cuando cae el sistema externo de facturación o bancos.
  - Tiempo de detección de la falla menor a 2 segundos mediante *health checks*.
  - 0 transacciones perdidas gracias al respaldo en el broker de mensajes y réplicas de base de datos.

**Escenario 2: ASR-02 — Rendimiento, Latencia y Escalabilidad Concurrente**

- **Atributo de Calidad:**Rendimiento y Escalabilidad.

- **Fuente del Estímulo:** Múltiples clientes B2C, portales B2B y aplicaciones móviles de transportistas concurrentes.

- **Estímulo:** Un peak de 15.000 solicitudes por segundo (RPS) consultando el seguimiento de envíos (*tracking*) y creando órdenes nuevas.

- **Artefacto Afectado:** Balanceador de carga, API Gateway, Servicio de Tracking (con Caché Redis y Réplicas de lectura) y Servicio de Órdenes.

- **Entorno Operacional:** Evento comercial de alta demanda (*CyberMonday*) con tráfico 10 veces mayor al promedio diario.

- **Respuesta Esperada del Sistema:** El balanceador de carga y el API Gateway reparten las solicitudes entre múltiples instancias de los servicios (escalamiento horizontal). Las consultas de tracking se responden directamente desde la caché en memoria (Redis) para no saturar la base de datos principal, mientras que la creación de órdenes se guarda rápidamente y delega el resto del flujo por eventos.

- **Medida Cuantitativa de la Respuesta:**
    - **Latencia en el percentil 95 (p95): < 250 ms** tanto para consultas de tracking como para creación de órdenes.
    - **Latencia en el percentil 99 (p99): < 450 ms**
    - **Tasa de errores de servidor(HTTP 5xx): < 0.01%** sosteniendo los **15.000 RPS** constantes.
    - **Tiempo de creación de nuevas instancias (escalamiento horizontal):** < 30 segundos.

## Escenario 3: ASR-03 — Mantenibilidad, Modificabilidad y Extensibilidad

- **Atributo de Calidad:** Mantenibilidad (Extensibilidad y Desacoplamiento).

- **Fuente del Estímulo:** Equipo de desarollo NexusCorp.

- **Estímulo:** Solicitud de conectar un nuevo operador logístico externo (ej. DHL) o una nueva pasarela de pago (ej. MercadoPago) con formatos de API diferentes.

- **Artefacto Afectado:** Servicio de Integración Logística o Servicio de Pagos.

- **Entorno Operacional:**Desarrollo y despliegue continuo con el sistema operando 24/7.

- **Respuesta Esperada del Sistema:** El equipo programa un nuevo conector aplicando interfaces estandarizadas y patrones de diseño (*Adapter* y *Factory Method*), desplegándolo en su propio microservicio sin modificar ni reiniciar el servicio central de órdenes.

- **Medida Cuantitativa de la Respuesta:**

    - **Tiempo total de desarrollo, pruebas y paso a producción: < 5 días hábiles** (<= 40 horas).

- **Código modificado en el núcleo de órdenes (*Order Core*):** 0 líneas modificadas (cumpliendo el Principio Abierto/Cerrado de SOLID).
- **Tiempo de inactividad por despliegue:** 0 minutos.

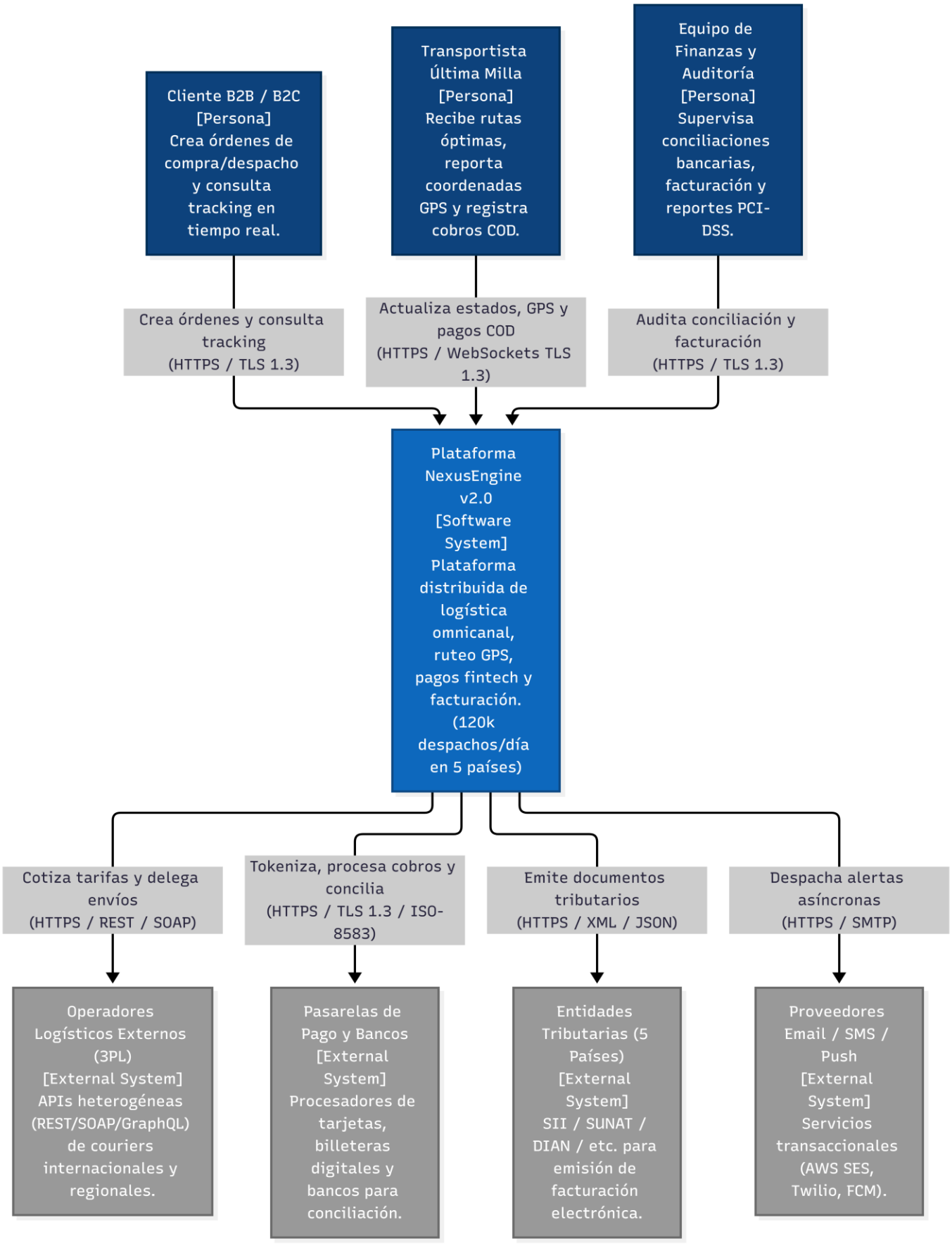
### Escenario 4: ASR-04 — Seguridad, Integridad Financiera y Cumplimiento PCI-DSS

- **Atributo de Calidad:** Seguridad (Confidencialidad, Integridad y Auditoría).
- **Fuente del Estímulo:** Atacante externo o usuario interno intentando leer datos de pago o modificar un registro contable.
- **Estímulo:** Intento de interceptar datos de tarjetas en la red o ejecutar un `UPDATE/DELETE` sobre los registros de conciliación bancaria.
- **Artefacto Afectado:** API Gateway, Servicio de Pagos y Base de Datos de Auditoría Financiera
- **Entorno Operacional:** Operación normal bajo normativa de seguridad PCI-DSS.
- **Respuesta Esperada del Sistema:** Toda comunicación exige cifrado TLS 1.3. Los datos sensibles de tarjetas se reemplazan por tokens cumpliendo PCI-DSS antes de procesarse. La base de datos de auditoría financiera funciona en modo de solo inserción (sin permisos de edición ni borrado) guardando una firma hash SHA-256 por cada transacción y enviando una alerta inmediata al sistema de monitoreo si alguien intenta alterarla.
- **Medida Cuantitativa de la Respuesta:**
    - **Datos de tarjetas expuestos en texto plano:** 0% (100% tokenizado bajo PCI-DSS y cifrado en reposo con AES-256).
    - **Tráfico cifrado con TLS 1.3:** 100% (rechazo inmediato de conexiones no seguras).
    - **Registros históricos alterables en auditoría:** 0 registros modificables.

## SECCIÓN 3: Representación de Arquitectura mediante Vistas (Opción B: Modelo C4 de Simon Brown)

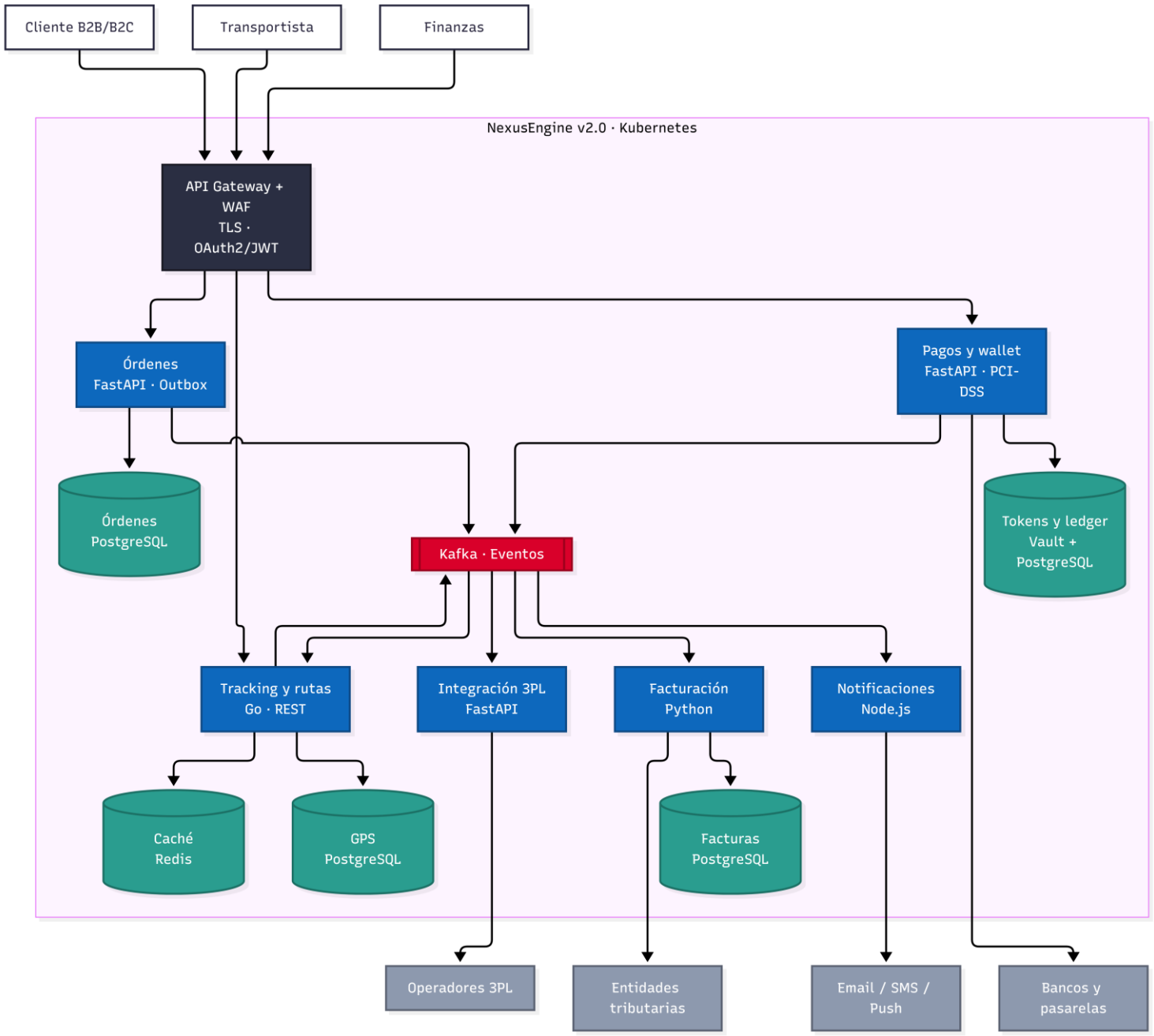
## 3.1. Nivel 1: Diagrama de Contexto del Sistema (C1)

Muestra cómo NexusEngine interactúa con los usuarios y sistemas externos dentro del ecosistema regional de 5 países.



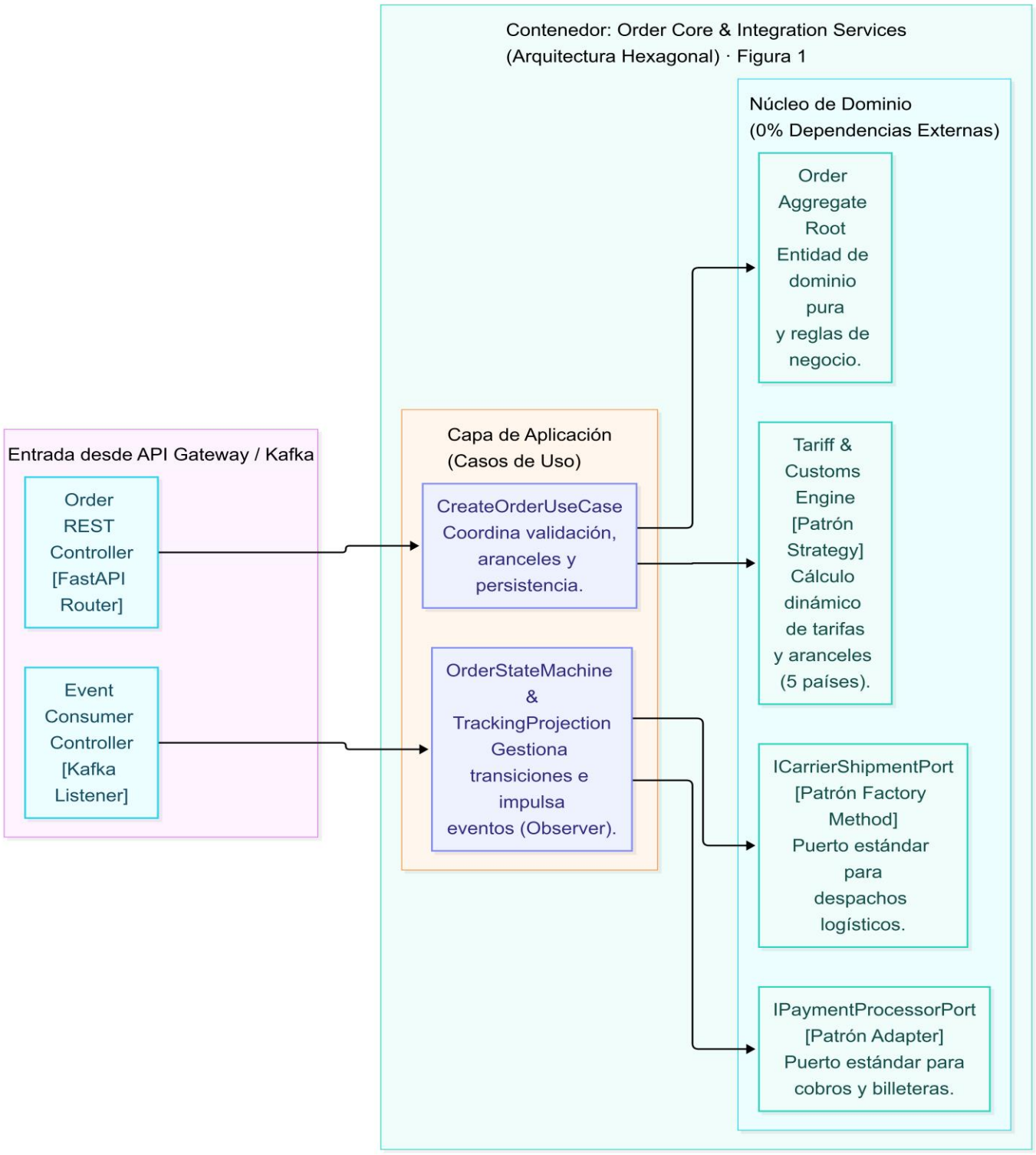
## 3.2. Nivel 2: Diagrama de Contenedores (C2)

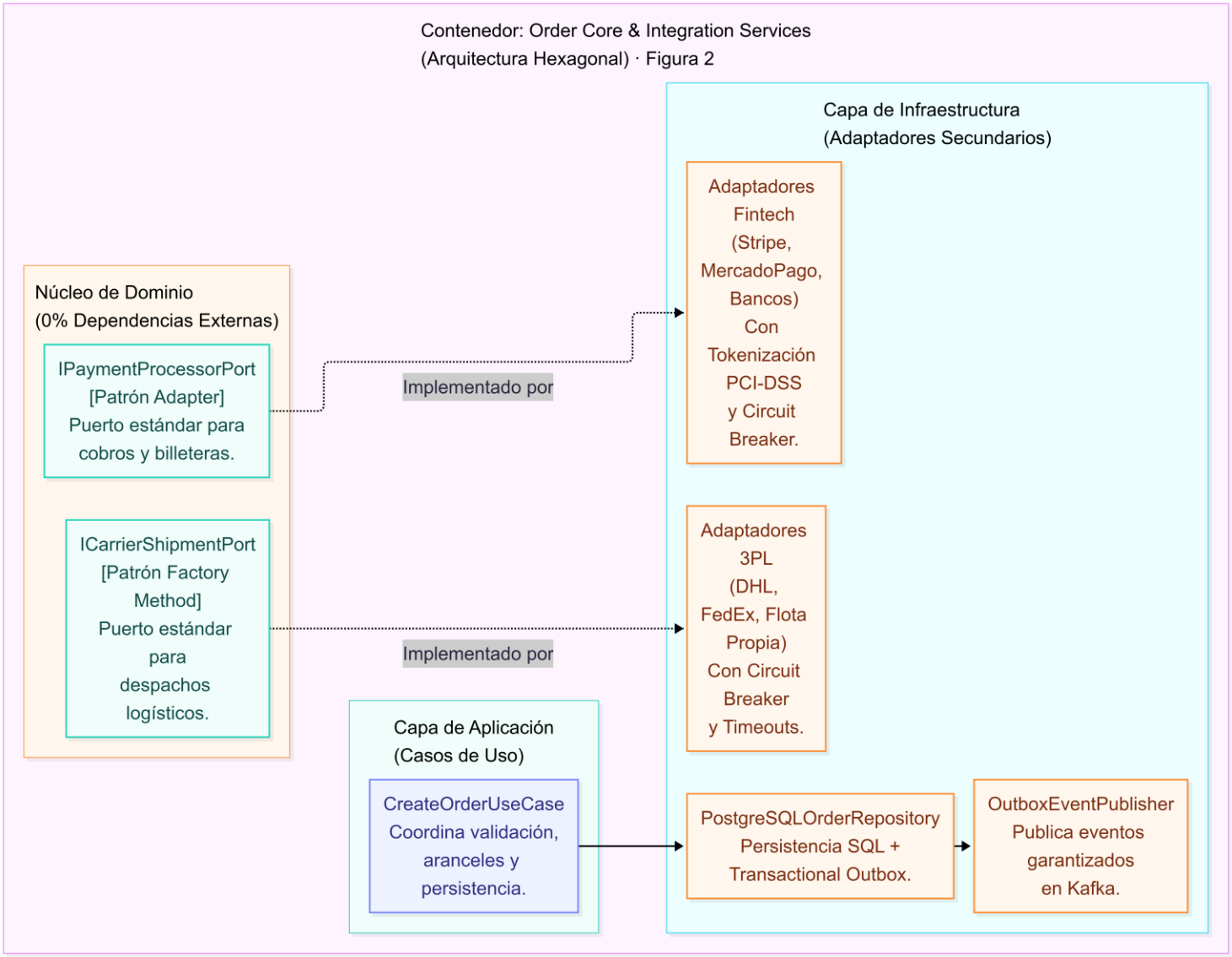
En esta vista abrimos el sistema para mostrar los microservicios, las bases de datos independientes por dominio y el bus de eventos en Apache Kafka que reemplazan al monolito original dentro del clúster de Kubernetes.



## 3.3. Nivel 3: Diagrama de Componentes (C3) — Descomposición del Antiguo `OrderManager`

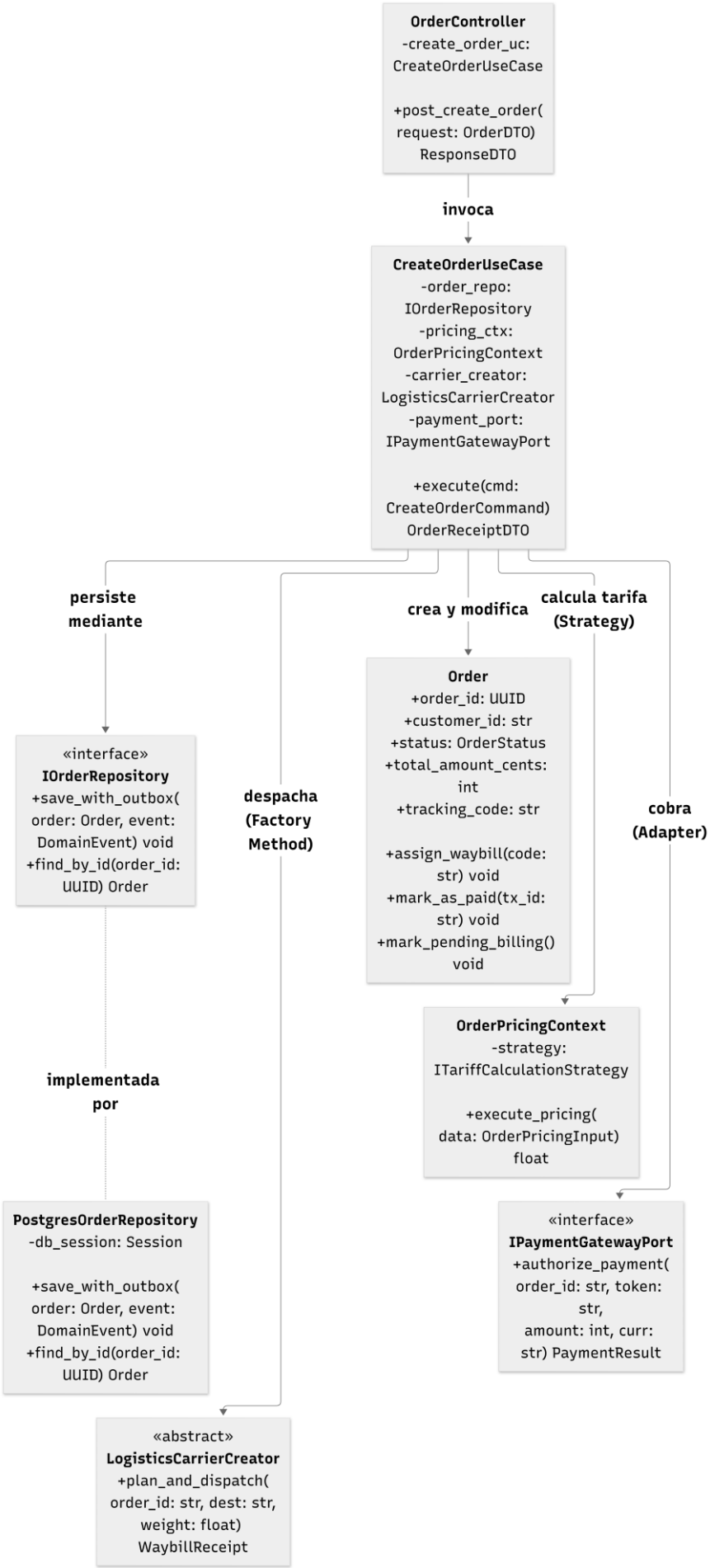
Este nivel detalla la arquitectura interna de los contenedores críticos (**Order Core Service**, **Carrier Integration Service** y **Payment Service**), demostrando cómo desarmamos el antiguo OrderManager (14.000 líneas de código) aplicando Arquitectura Hexagonal (Puertos y Adaptadores) y el Principio de Responsabilidad Única (SRP).






### 3.4. Nivel 4: Diagrama de Clases Detallado (C4)

Finalmente, este diagrama muestra el diseño de clases interno del servicio principal de órdenes (Order Core), detallando cómo el caso de uso CreateOrderUseCase coordina la entidad Order, el repositorio y las interfaces de los patrones GoF que detallamos en la siguiente sección.


OrderController
-create_order_uc: CreateOrderUseCase
+post_create_order( request: OrderDTO) ResponseDTO
invoca
CreateOrderUseCase
-order_repo: IOrderRepository
-pricing_ctx: OrderPricingContext
-carrier_creator: LogisticsCarrierCreator
-payment_port: IPaymentGatewayPort
+execute(cmd: CreateOrderCommand) OrderReceiptDTO
persiste mediante
crea y modifica
calcula tarifa (Strategy)
despacha (Factory Method)
cobra (Adapter)
«interface»
IOrderRepository
+save_with_outbox( order: Order, event: DomainEvent) void
+find_by_id(order_id: UUID) Order
Order
+order_id: UUID
+customer_id: str
+status: OrderStatus
+total_amount_cents: int
+tracking_code: str
+assign_waybill(code: str) void
+mark_as_paid(tx_id: str) void
+mark_pending_billing() void
OrderPricingContext
-strategy: ITariffCalculationStrategy
+execute_pricing( data: OrderPricingInput) float
implementada por
PostgresOrderRepository
-db_session: Session
+save_with_outbox( order: Order, event: DomainEvent) void
+find_by_id(order_id: UUID) Order
«interface»
IPaymentGatewayPort
+authorize_payment( order_id: str, token: str, amount: int, curr: str) PaymentResult
«abstract»
LogisticsCarrierCreator
+plan_and_dispatch( order_id: str, dest: str, weight: float) WaybillReceipt

# SECCIÓN 4: Diseño Detallado con Patrones de Diseño Gang of Four (GoF)

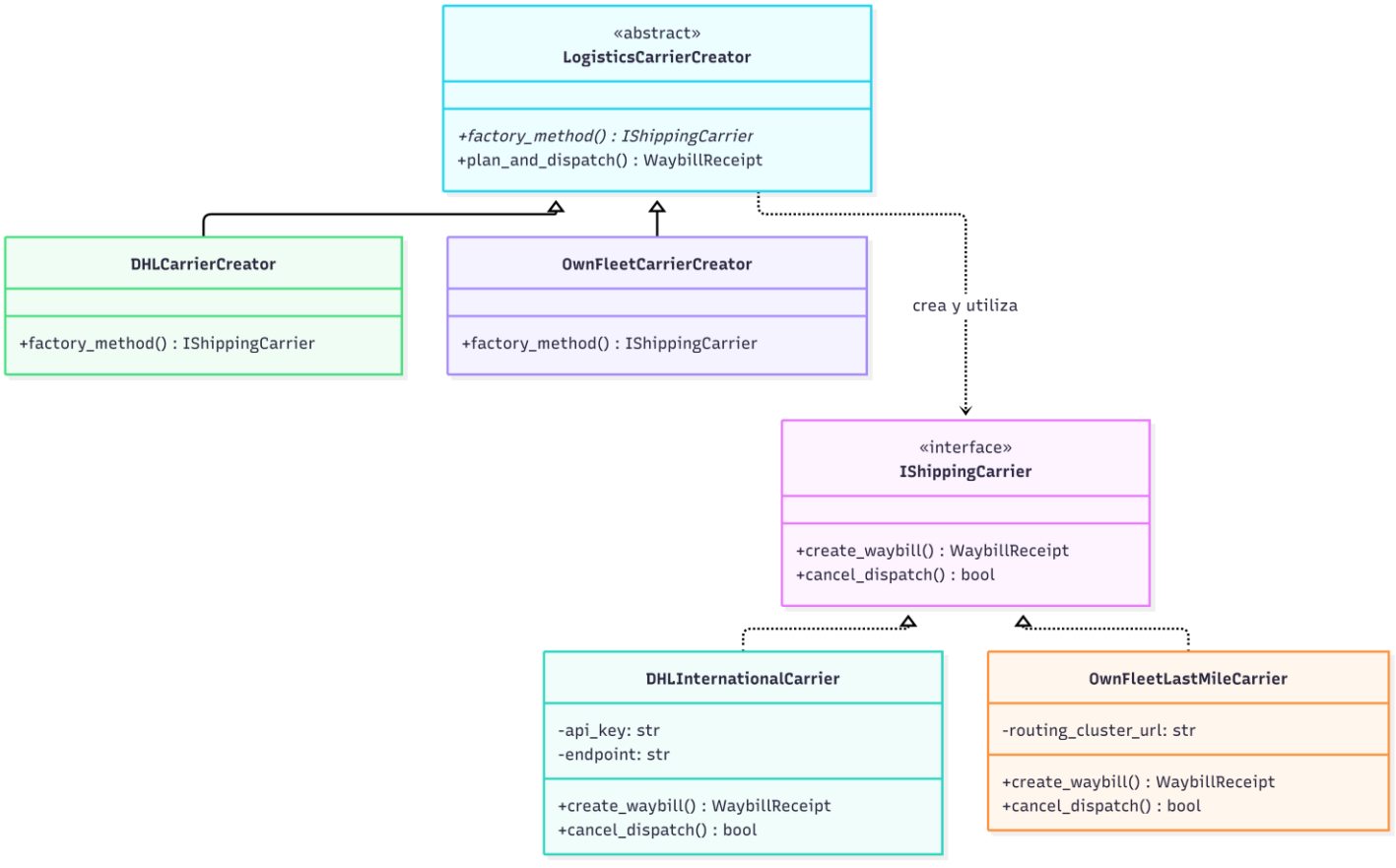
Para resolver la rigidez de integraciones externas (ASR-03), el acoplamiento del OrderManager (deuda técnica) y la asincronía de eventos (ASR-01/ASR-02), se implementan tres patrones GoF de distintas categorías en **Python 3.11+** utilizando tipado estático estricto (abc, typing, dataclasses).

## 4.1. Patrón Creacional: Factory Method (Instanciación Desacoplada de Operadores Logísticos)

### A. Problema que resuelve en NexusCorp

En el monolito original, agregar un nuevo operador de transporte internacional requería modificar bloques if/elif/else gigantes dentro de OrderManager y redesplegar toda la aplicación. Aplicando **Factory Method**, el núcleo transaccional trabaja únicamente contra las abstracciones LogisticsCreator (Creator) e IShippingCarrier (Product). Incorporar un nuevo courier para un nuevo país toma menos de 5 días hábiles (ASR-03) simplemente registrando una nueva subclase creadora sin tocar el código existente (cumpliendo el Principio Open/Closed).

### *B. Diagrama de Clases UML Formal (Factory Method)*




**Parámetros:** create_waybill y plan_and_dispatch reciben order_id: str, destination: str y weight_kg: float; cancel_dispatch recibe tracking_code: str.

### C. Implementacion en Código (Python)

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
from datetime import datetime
from typing import Dict, Type

@dataclass(frozen=True)
class WaybillReceipt:
    tracking_code: str
    carrier_name: str
    estimated_delivery: datetime
    label_url: str

# 1. Interfaz del Producto
class IShippingCarrier(ABC):
    @abstractmethod
    def create_waybill(self, order_id: str, destination: str, weight_kg: float) ->
WaybillReceipt:
        pass

    @abstractmethod
    def cancel_dispatch(self, tracking_code: str) -> bool:
        pass

# 2. Productos Concretos
class DHLInternationalCarrier(IShippingCarrier):
    def create_waybill(self, order_id: str, destination: str, weight_kg: float) ->
WaybillReceipt:
        return WaybillReceipt(
            tracking_code=f"DHL-INT-{order_id}",
            carrier_name="DHL Express International",
            estimated_delivery=datetime.now(),
            label_url=f"https://api.dhl.com/labels/{order_id}.pdf"
        )

    def cancel_dispatch(self, tracking_code: str) -> bool:
        return True

class OwnFleetLastMileCarrier(IShippingCarrier):
    def create_waybill(self, order_id: str, destination: str, weight_kg: float) ->
WaybillReceipt:
        return WaybillReceipt(
            tracking_code=f"NEXUS-LM-{order_id}",
            carrier_name="NexusCorp Flota Propia",
            estimated_delivery=datetime.now(),
            label_url=f"https://internal.nexuscorp.com/qr/{order_id}.png"
        )

    def cancel_dispatch(self, tracking_code: str) -> bool:
        return True

# 3. Clase Creadora Abstracta (Creator)
class LogisticsCarrierCreator(ABC):
    @abstractmethod
    def factory_method(self) -> IShippingCarrier:
        pass

    def plan_and_dispatch(self, order_id: str, destination: str, weight_kg: float) ->
WaybillReceipt:
        carrier = self.factory_method()
        return carrier.create_waybill(order_id, destination, weight_kg)
```

```python
# 4. Creadores Concretos
class DHLCarrierCreator(LogisticsCarrierCreator):
    def factory_method(self) -> IShippingCarrier:
        return DHLInternationalCarrier()

class OwnFleetCarrierCreator(LogisticsCarrierCreator):
    def factory_method(self) -> IShippingCarrier:
        return OwnFleetLastMileCarrier()

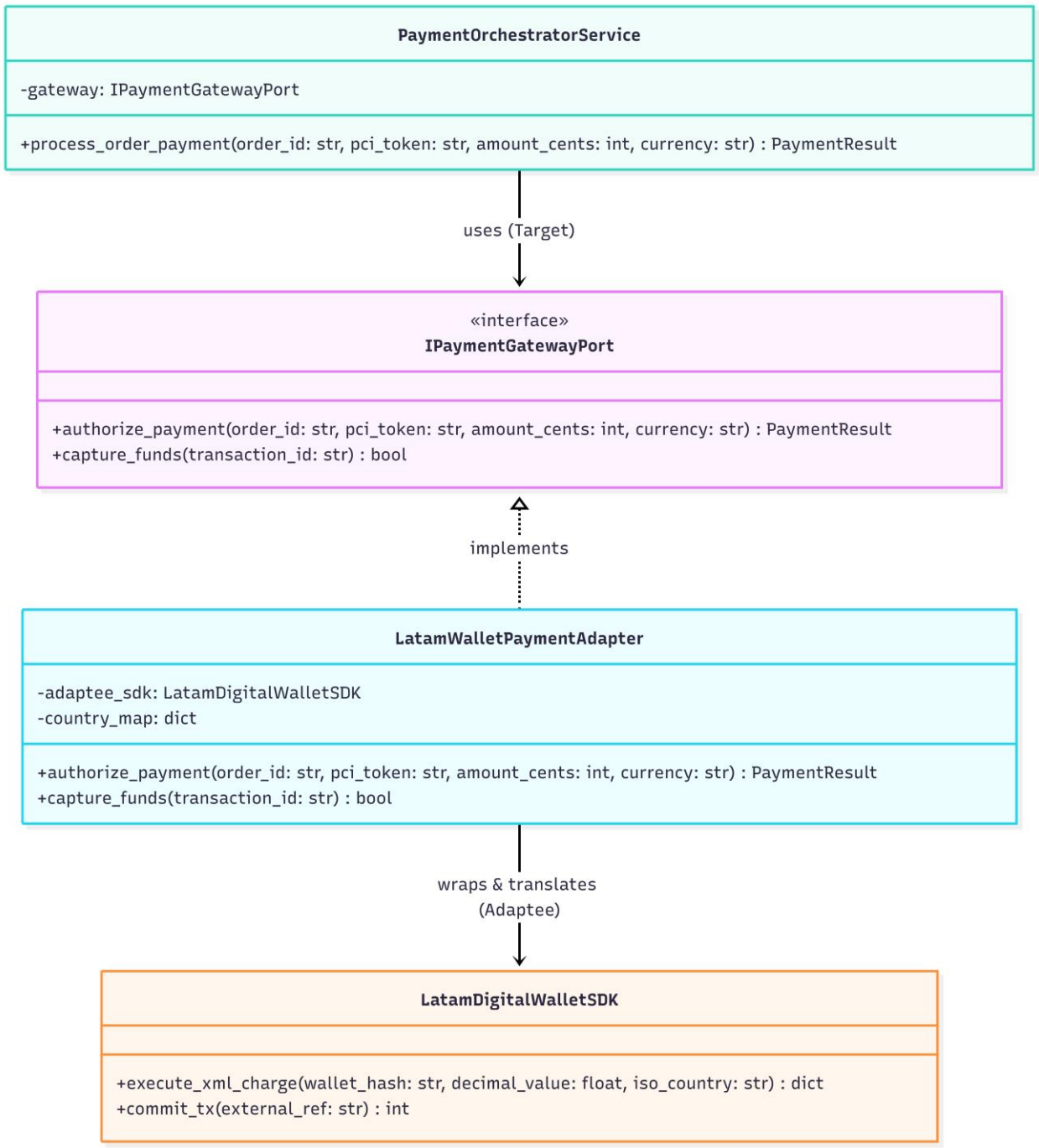
# Registro de proveedores para evitar bloques if/else acoplados (ASR-03)
CARRIER_REGISTRY: Dict[str, Type[LogisticsCarrierCreator]] = {
    "INTERNATIONAL_DHL": DHLCarrierCreator,
    "LOCAL_LAST_MILE": OwnFleetCarrierCreator,
}
```

## 4.2. Patrón Estructural: Adapter (Homologación de Pasarelas de Pago y Billeteras Fintech)

### *A. Problema que resuelve en NexusCorp*

NexusCorp opera en 5 países e incorpora billeteras digitales, pasarelas bancarias y pagos contra entrega (COD), cada una con contratos incompatibles (JSON anidados, códigos ISO-8583, XML o SDKs propietarios) y exigencias estrictas de tokenización PCI-DSS (ASR-04). El patrón **Adapter** envuelve el SDK/API de cada proveedor externo (`Adaptee`) y lo traduce a una interfaz de dominio única y segura (`IPaymentGatewayPort`), impidiendo que detalles propietarios contaminen el microservicio de pagos.

### B. Diagrama de Clases UML Formal (Adapter)




### C. Implementación en Código (Python)

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
import hashlib

@dataclass(frozen=True)
class PaymentResult:
    is_authorized: bool
    transaction_id: str
    audit_signature_sha256: str

# 1. Interfaz Target (Estándar interno de NexusCorp bajo PCI-DSS)
class IPaymentGatewayPort(ABC):
    @abstractmethod
    def authorize_payment(
        self, order_id: str, pci_token: str, amount_cents: int, currency: str
```

```
    ) -> PaymentResult:
        pass

    @abstractmethod
    def capture_funds(self, transaction_id: str) -> bool:
        pass

# 2. Adaptee (SDK externo incompatible de billetera digital)
class LatamDigitalWalletSDK:
    def execute_xml_charge(self, wallet_hash: str, decimal_value: float, iso_curr: str) -> dict:
        return {"status_code": 200, "ext_tx_ref": "WALLET-TX-998877"}

    def commit_tx(self, external_ref: str) -> int:
        return 1

# 3. Adapter (Traduce llamadas del dominio al SDK externo)
class LatamWalletPaymentAdapter(IPaymentGatewayPort):
    def __init__(self, sdk: LatamDigitalWalletSDK) -> None:
        self._sdk = sdk

    def authorize_payment(
        self, order_id: str, pci_token: str, amount_cents: int, currency: str
    ) -> PaymentResult:
        decimal_amount = round(amount_cents / 100.0, 2)
        raw_response = self._sdk.execute_xml_charge(
            wallet_hash=pci_token,
            decimal_value=decimal_amount,
            iso_curr=currency
        )
        authorized = raw_response.get("status_code") == 200
        tx_id = raw_response.get("ext_tx_ref", "FAILED")

        # Firma SHA-256 para el registro inmutable de auditoría (ASR-04)
        payload = f"{order_id}:{tx_id}:{amount_cents}:{currency}"
        signature = hashlib.sha256(payload.encode("utf-8")).hexdigest()

        return PaymentResult(
            is_authorized=authorized,
            transaction_id=tx_id,
            audit_signature_sha256=signature
        )

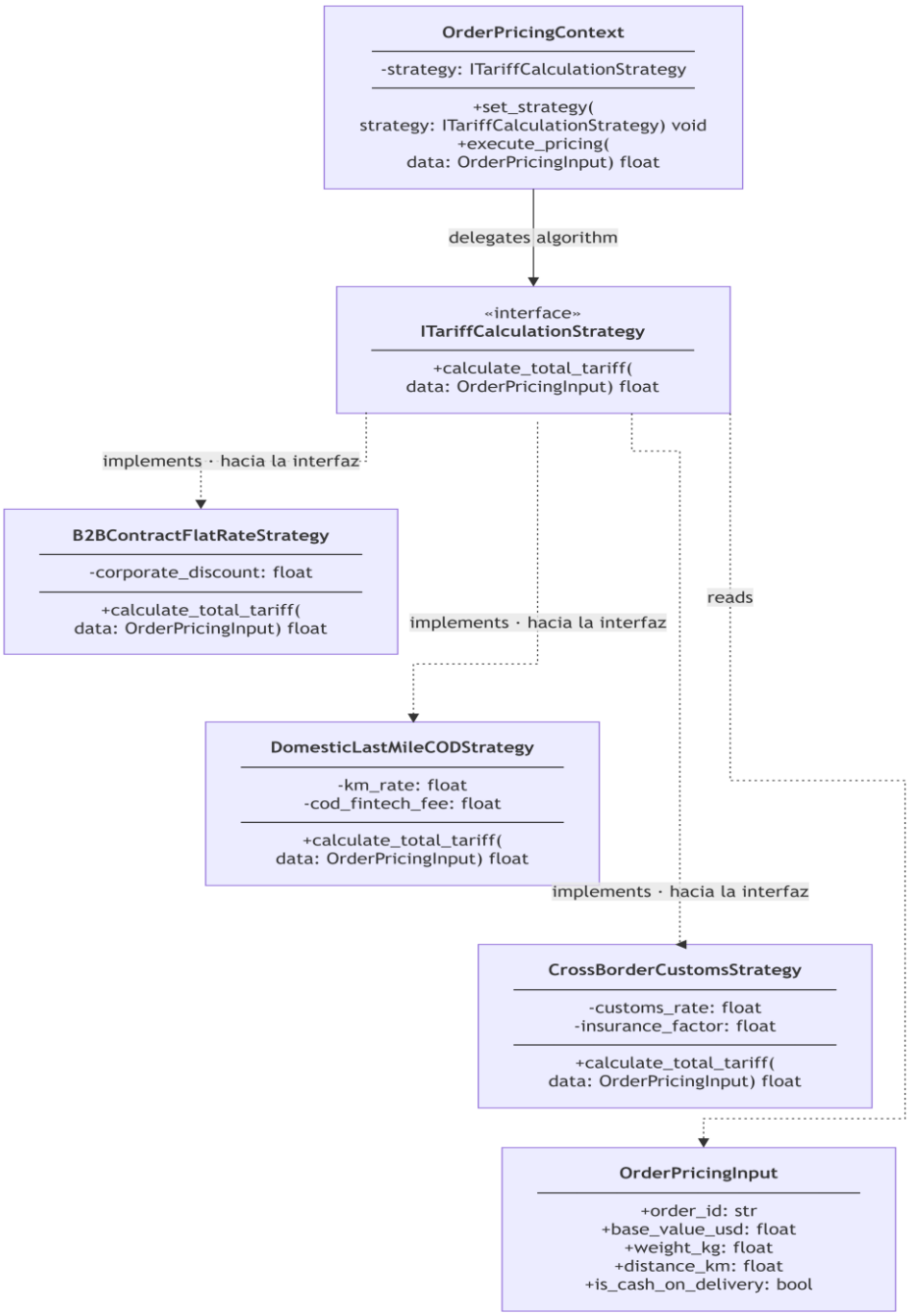
    def capture_funds(self, transaction_id: str) -> bool:
        return self._sdk.commit_tx(transaction_id) == 1
```

## 4.3. Patrón de Comportamiento: Strategy (Cálculo Dinámico de Aranceles, Impuestos y Tarifas de Ruta)

### *A. Problema que resuelve en NexusCorp*

El antiguo OrderManager mezclaba cálculos de aranceles aduaneros de 5 países, recargos por cadena de frío, tarifas dinámicas de última milla y comisiones de pago contra entrega (COD) en un mismo método monolítico. Con el patrón **Strategy**, cada algoritmo de tarificación y cálculo tributario se encapsula en una clase independiente que implementa ITariffCalculationStrategy. El contexto (OrderPricingContext) selecciona e inyecta la estrategia en tiempo de ejecución según el país, tipo de cliente (B2B/B2C) o modalidad de envío

## B. Diagrama de Clases UML Formal (Strategy)




## C. Implementación en Código (Python)

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass(frozen=True)
class OrderPricingInput:
    order_id: str
    base_value_usd: float
    weight_kg: float
    distance_km: float
    is_cash_on_delivery: bool

# 1. Interfaz Strategy
class ITariffCalculationStrategy(ABC):
```

```python
    @abstractmethod
    def calculate_total_tariff(self, data: OrderPricingInput) -> float:
        pass

# 2. Estrategias Concretas
class CrossBorderCustomsStrategy(ITariffCalculationStrategy):
    def calculate_total_tariff(self, data: OrderPricingInput) -> float:
        customs_duty = data.base_value_usd * 0.18
        weight_surcharge = data.weight_kg * 4.50
        international_freight = data.distance_km * 0.12
        return round(customs_duty + weight_surcharge + international_freight, 2)

class DomesticLastMileCODStrategy(ITariffCalculationStrategy):
    def calculate_total_tariff(self, data: OrderPricingInput) -> float:
        base_delivery = 3.50 + (data.distance_km * 0.25) + (data.weight_kg * 0.80)
        cod_fee = (data.base_value_usd * 0.025) if data.is_cash_on_delivery else 0.0
        return round(base_delivery + cod_fee, 2)

class B2BContractFlatRateStrategy(ITariffCalculationStrategy):
    def calculate_total_tariff(self, data: OrderPricingInput) -> float:
        return round((data.weight_kg * 0.50) + (data.distance_km * 0.10), 2)

# 3. Contexto de Tarificación
class OrderPricingContext:
    def __init__(self, strategy: ITariffCalculationStrategy) -> None:
        self._strategy = strategy

    def set_strategy(self, strategy: ITariffCalculationStrategy) -> None:
        self._strategy = strategy

    def execute_pricing(self, data: OrderPricingInput) -> float:
        return self._strategy.calculate_total_tariff(data)
```

# SECCIÓN 5: Registro de Decisiones de Arquitectura (ADRs)

A continuación se documentan formalmente tres (3) **Architectural Decision Records (ADRs)** que sustentan las decisiones estructurales más críticas del rediseño:

## ADR-001: Paso de Monolito a Microservicios con Comunicación por Eventos (EDA)

- **Estado:** Aceptado
- **Contexto del Problema:** Actualmente NexusEngine funciona como un monolito en Ruby/PostgreSQL donde la clase `OrderManager` (14.000 líneas) concentra catálogo, rutas GPS, pagos y facturación. Al estar todo fuertemente acoplado, un fallo en facturación o en un banco externo detiene el despacho de pedidos en ruta, lo que causó 4 horas de caída en CyberMonday y pérdidas por USD 350.000. Se requiere alcanzar un 99.95% de disponibilidad (ASR-01).

- **Alternativas Evaluadas:** 1) Escalar verticalmente el servidor del monolito actual; 2) Separar en microservicios comunicados de forma síncrona; 3) Separar en microservicios comunicados de forma asíncrona mediante un broker de eventos (EDA).

- **Decisión Tomada:**

1. Desacoplar el monolito en microservicios independientes (*Órdenes*, *Tracking y Rutas*, *Integración 3PL*, *Pagos* y *Facturación*) desplegados con redundancia (múltiples instancias) y comunicados mediante eventos usando Apache Kafka como broker de mensajería.

**Consecuencias Técnicas:**

- **Positivas (Beneficios):**
    - **Tolerancia a fallos y alta disponibilidad (ASR-01):** Si el módulo de facturación o un banco falla, los servicios de órdenes y rutas siguen operando sin interrupciones, cumpliendo el SLA del 99.95%..
    - **Mantenibilidad y despliegues independientes (ASR-03):** Permite actualizar conectores o reglas de facturación sin reiniciar toda la plataforma.

- **Negativas (Trade-offs):**
    - **Mayor complejidad de operación y monitoreo:** Requiere administrar múltiples servicios, contenedores y un broker de eventos en lugar de una sola aplicación.
    - **Procesamiento asíncrono:** La facturación se completa unos segundos después de creado el pedido a través de la cola de mensajes, por lo que el sistema maneja estados intermedios.

## ADR-002: Incorporación de Caché en Memoria (Redis) y Réplicas de Lectura para Rendimiento

- **Estado:** Aceptado

- **Contexto del Problema:** El requisito ASR-02 exige soportar picos de 15.000 solicitudes por segundo (RPS) con un tiempo de respuesta menor a 250 ms (p95) para consultas de seguimiento (*tracking*) y creación de órdenes. Consultar directamente la base de datos transaccional PostgreSQL en cada solicitud de seguimiento genera bloqueos y alta latencia durante campañas masivas.
- **Alternativas Evaluadas:** 1) Mantener todas las lecturas y escrituras sobre una única base de datos PostgreSQL; 2) Incorporar una capa de caché en memoria con Redis junto a réplicas de lectura de PostgreSQL y escalamiento horizontal.
- **Decisión Tomada:** Implementar una caché distribuida en memoria (Redis) que almacena el estado actual de los envíos actualizada por eventos desde Kafka, apoyada por réplicas de lectura en PostgreSQL y balanceo de carga entre múltiples instancias del servicio de Tracking.

**Consecuencias Técnicas y Trade-offs:**

- **Positivas (Beneficios):**

- **Cumplimiento del tiempo de respuesta (ASR-02):** Las consultas de tracking leen directamente desde Redis respondiendo entre 10 ms y 25 ms (muy por debajo de los 250 ms requeridos).
- **Desahogo de la base de datos principal:** La base de datos de órdenes queda libre para procesar nuevas compras rápidamente.

- **Negativas (Trade-offs):**
  - **Gestión de actualización e invalidación de caché:** Obliga a actualizar la caché cada vez que un transportista reporta un cambio de ubicación, existiendo un desfase menor a medio segundo que no afecta al usuario final
  - **Mayor consumo de memoria RAM:** Implica el costo adicional de mantener los servidores de caché en alta disponibilidad.

## ADR-003: Uso de API Gateway, Tokenización y Base de Datos de Auditoría para Seguridad (PCI-DSS)

- **Estado:** Aceptado
- **Contexto del Problema:**El requisito ASR-04 exige proteger los pagos digitales y cobros contra entrega en los 5 países mediante cifrado TLS 1.3 en tránsito, AES-256 en reposo, tokenización bajo norma PCI-DSS y auditoría inmutable de transacciones financieras
- **Alternativas Evaluadas:** 1) Validar la seguridad por separado dentro de cada microservicio; 2) Centralizar el acceso mediante un API Gateway + WAF y aislar los cobros en un servicio de pagos con base de datos de auditoría de solo inserción.

**Decisión Tomada:**

1. Ubicar un API Gateway con Firewall de Aplicaciones (WAF) en la entrada para forzar conexiones TLS 1.3 y autenticación OAuth 2.0 / JWT. Además, aislar el Servicio de Pagos para que reemplace los datos de tarjetas por tokens PCI-DSS y registre cada movimiento contable en una tabla de auditoría donde solo se permiten inserciones (`INSERT`), bloqueando ediciones o borrados y asegurando cada fila con un hash SHA-256.

**Consecuencias Técnicas y Trade-offs:**

- **Positivas (Beneficios):**
  - **Protección de datos y cumplimiento PCI-DSS (ASR-04):** Los servicios de pedidos y logística nunca tocan datos reales de tarjetas, reduciendo el riesgo de filtraciones.
  - **Trazabilidad financiera garantizada:** Ningún usuario ni administrador puede alterar o borrar un registro de pago histórico sin que se detecte la rotura del hash SHA-256.

- **Negativas (Trade-offs):**
  **Pequeña sobrecarga por cifrado:** La validación de tokens y el cifrado suman cerca de 12 ms al procesar el pago.
  - **Correcciones mediante nuevos registros:** Al estar prohibido hacer `UPDATE` o `DELETE` en la auditoría, cualquier anulación o reembolso debe registrarse insertando una nueva fila de reverso contable.

# SECCIÓN 6: Matriz de Trazabilidad Arquitectónica

Esta matriz demuestra cómo cada problemática del negocio y cada Requerimiento ASR es resuelto y verificado por las vistas C4, los patrones GoF y las decisiones arquitectónicas (ADRs):

| ID ASR / Atributo | Objetivo de Negocio / Deuda Técnica que Resuelve | Escenario SEI Asociado | Contenedores y Componentes (Vistas C4) | Patrones GoF y de Arquitectura Aplicados | Métrica de Verificación Cuantitativa | ADR que lo Sustenta |
|---|---|---|---|---|---|---|
| **ASR-01**<br><br>Disponibilidad y Resiliencia | Evitar caídas totales en CyberMonday (pérdida de USD 350k) cuando falla facturación o bancos. | **Escenario 1:** Tolerancia a fallos ante caída de pasarela bancaria o nodo de facturación | **C2:** Broker de Eventos Kafka, Servicio de Órdenes, Servicio de Facturación<br>**C3:** Publicador de Eventos y Adaptadores con tolerancia a fallos. | • **Observer** (Notificación de eventos asíncronos)<br><br>• Redundancia y aislamiento de fallas | • **SLA >= 99.95%** anual<br>• **0%** órdenes detenidas por caída en facturación<br>• **0** mensajes perdidos | **ADR-001** (Microservicios y EDA con Kafka) |
| **ASR-02**<br><br>Rendimiento y Escalabilidad | Soportar crecimiento regional (120k despachos/día en 5 países) y alta concurrencia sin bloqueos de BD | **Escenario 2:** Pico concurrente de 15.000 RPS en consultas de tracking y creación de órdenes. | **C2:** Balanceador, API Gateway, Servicio de Tracking, Caché Redis y BD PostgreSQL.<br>**C3:** Proyección de Tracking | • **Caché en memoria y Réplicas de lectura**<br><br>•**Escalamiento horizontal** | • **15.000 RPS** sostenidos<br>• Latencia **p95 < 250 ms**<br>• Tasa de error 5xx **< 0.01%**. | **ADR-002** (Caché Redis y Réplicas)<br><br>**ADR-001** (Procesamiento asíncrono) |
| **ASR-03**<br><br>Mantenibilidad y Extensibilidad | Eliminar el *God Object* (`OrderManager` de 14.000 líneas) y la rigidez al integrar proveedores externos. | **Escenario 3:** Integración de nuevo operador logístico 3PL o pasarela de pago sin tocar el núcleo. | **C2:** Servicio Integración 3PL, Servicio de Pagos.<br>**C3/C4:** ICarrierShipmentPort, IPaymentGatewayPort, TariffEngine. | • **Factory Method** (Creacional: Operadores 3PL)<br>• **Adapter** (Estructural: Pasarelas de Pago)<br>• **Strategy** (Comportamiento: Tarifas y Aduanas) | • Integración de nuevo proveedor en **< 5 días hábiles**.<br>• **0 líneas** modificadas en `Order Core`.<br>• **0 min** de baja en despliegue. | **ADR-001** (Separación en microservicios y Arquitectura Hexagonal) |
| **ASR-04**<br><br>Seguridad e Integridad Financiera | Proteger pagos digitales y cobros en ruta en 5 países y asegurar conciliación bancaria auditable. | **Escenario 4:** Protección de credenciales de pago y bloqueo de alteraciones en registros contables. | **C2:** API Gateway + WAF (TLS 1.3), Servicio de Pagos aislado, Bóveda de Tokens y BD de Auditoría. | • **Adapter** (Tokenización y firma SHA-256 al integrar pasarelas) *Ledger*<br>• Registro de auditoría de solo inserción | • **100%** tráfico TLS 1.3 y reposo AES-256.<br>• **0%** exposición de tarjetas (PCI-DSS).<br>• **0** registros alterables sin detección. | **ADR-003** (API Gateway, Tokenización PCI-DSS y Auditoría Inmutable) |